



CSI142 Object-Oriented Programming

Week 1, Lecture 2: Java Architecture, I/O, Control Flow

Wednesday, 28 Jan 2026 • 08:00-09:00 • Room 252-008 • Semester II 2025/26



Quick check-in (raise your hand):

- I can compile and run a Java program
- VS Code is working with Java 17
- I can read input using Scanner

Today

- Java toolchain and file structure
- Console output and input patterns
- Control flow refresher: if, for, while
- Common pitfalls and debugging habits
- Mini practice task



How your Java code becomes a running program





```
CSI142/  
  week1/  
    HelloCSI142.java  
  week2/  
    SwitchDemo.java  
  assignments/  
    a1/  
      src/  
      README.md
```

- Keep one folder per week, and one per assignment
- Name files clearly, avoid spaces in folder names
- Back up with Git (push regularly)



```
double price = 12.5;  
int qty = 3;  
System.out.println("Total = " + (price * qty));  
System.out.printf("Total = %.2f\n", price * qty);
```

- println is great for quick output
- printf gives formatted output (useful for reports)
- **Notice:** parentheses matter in string concatenation



```
import java.util.Scanner;

Scanner in = new Scanner(System.in);
System.out.print("Enter age: ");
int age = in.nextInt();
in.nextLine(); // consume end-of-line

System.out.print("Enter name: ");
String name = in.nextLine();
```

- Mixing **nextInt** and **nextLine** requires care
- Option: read full line, then parse
- Close Scanner at end of program (not inside loops)



```
int number;
while (true) {
    System.out.print("Enter an integer: ");
    if (in.hasNextInt()) {
        number = in.nextInt();
        in.nextLine();
        break;
    } else {
        System.out.println("Not an integer, try again.");
        in.nextLine();
    }
}
```

- **hasNextInt** checks before reading
- The loop is a real example of a **sentinel pattern**
- Later we will also use **try-catch** for invalid input



- Use parentheses to make intent clear: `(age >= 18) && (citizen)`
- `&&` and `||` short-circuit, right side may not run
- Do not compare strings with `==`, use `.equals`
- Be careful with `=` vs `==` in conditions



Loops: choose the right tool

CSI142 • Sem II 2025/26

Pattern	Best for	Template idea
Counter-controlled	Known number of repeats	<code>for (int i=0; i<n; i++) { ... }</code>
Sentinel-controlled	Repeat until a condition	<code>while (!done) { ... }</code>
Input validation	Repeat until valid input	<code>do { ... } while (!valid);</code> (next lecture)



Write a small program:

- 1) Ask the user for a positive integer n
- 2) Print the sum $1 + 2 + \dots + n$
- 3) If the user enters a non-positive number, ask again

Hint: use a validation loop + a for-loop.



- Off-by-one errors in loops (start/stop conditions)
- Forgetting to update loop variables, causing infinite loops
- Scanner newline issues (nextInt then nextLine)
- String comparison: use `.equals`
- Ignoring compiler errors, always read the first error carefully



- Before Friday: practise Scanner input, if/else, for, while
- Bring one confusing error message to discuss (we will unpack it together)
- Friday: short knowledge check, then switch and do-while
- Lab: continue exercises, ask TAs for help with setup and debugging